Software Test Plan (STP) – UATC

Project: UATC – Universal Adaptive Transform Compression

Version: 1.0

Authors: Team UATC – S Chakshu, Sai Rishi Gangarapu (Team Lead), Sampada K, Manasa K

Date: 29-09-2026

Status: Draft

Project Guide: Dr. Sapna V M

1. Introduction

Purpose: This document defines the test plan for UATC v1.0. It sets out objectives, scope, strategy, environment, schedule, responsibilities and the test cases used to validate the requirements in the SRS and the design in the SAD.

Scope: Testing covers file and directory compression, extraction, adaptive per-block transform selection, the store decision, entropy coding, integrity verification, parallel execution, archive inspection, error handling, the CLI, the HTTP API and the web frontend. It also covers the non-functional requirements (correctness, performance, reliability, usability, maintainability, portability, observability, responsiveness) and security validation. Core operating-system behaviour and third-party library internals are excluded.

References: UATC SRS v1.0, UATC SAD v1.0, Project Synopsis, PES University SE Mini Project Guidelines (UE24CS341A).

Definitions: UATC, block, transform, store, CRC32, SHA-256, CLI, SRS, SAD, STP, RTM (see SAD Section 1.4). Requirement IDs FR-01 to FR-15 are from SRS Section 3.1. The SRS lists non-functional requirements by category only, so this STP labels them NFR-01 to NFR-08 (Section 3) for traceability; the same labels should be added to the SRS traceability matrix.

2. Test Items

- File analyzer and file-system layer
- Block Manager and archive index
- Archive Builder and Archive Reader (.uatc format)
- Compressibility Probe
- Adaptive Transform Engine (None, Delta, XOR, Byte-shuffle, RLE) and Transform Registry
- Entropy Coding Engine
- Integrity Module (CRC32, SHA-256)
- Thread-Pool Scheduler and parallel block processing
- Archive Inspector and Verifier (info, verify)
- CLI (compress, extract, info, verify, benchmark)
- API / Application Layer and Job Manager (HTTP)
- Web Frontend
- Benchmark harness

3. Features to be Tested

Features are mapped to SRS requirement IDs.

ID	Feature / requirement
FR-01	File and directory input
FR-02	Archive creation
FR-03	Extraction, byte-for-byte
FR-04	Per-block statistics
FR-05	Compressibility probe
FR-06	Adaptive transform selection
FR-07	Entropy coding
FR-08	Store-or-compress decision
FR-09	Integrity verification
FR-10	Parallel execution, identical output
FR-11	Archive inspection
FR-12	Error detection and reporting
FR-13	Web-based interface
FR-14	Progress and status display
FR-15	Frontend-backend API
NFR-01	Correctness: byte-for-byte reconstruction
NFR-02	Performance: throughput scales with threads; honest comparison with gzip/Zstandard
NFR-03	Reliability: corruption always detected
NFR-04	Usability: simple CLI and web UI
NFR-05	Maintainability: modular, add a transform without redesign
NFR-06	Portability: standard C++17
NFR-07	Observability: ratio, time, throughput, per-block transform
NFR-08	Responsiveness: UI usable during long jobs

4. Features Not to be Tested

- Internal correctness of third-party libraries (HTTP server, JSON library, gzip and Zstandard used only as benchmark baselines)
- Operating-system file system, disk and browser internals
- Lossy compression, audio/video codecs, native desktop application, distributed execution (out of scope in SRS 1.2)
- Absolute compression ratio targets: UATC does not claim to beat Zstandard, so ratio is reported, not pass/failed
- Multi-user access control (v1 is a local, single-user tool)

5. Test Approach / Strategy

Levels

- Unit tests (GoogleTest): each module tested in isolation by its owner. Code and branch coverage is measured with gcov/lcov.
- Integration tests: pipeline stages together, archive writer with reader, API with core, using SHA-256 to confirm lossless round trips.
- System tests: end-to-end use cases UC-1 to UC-5 through the CLI and the web UI.
- Acceptance tests: demo scenarios agreed with the project guide, run against the success criteria in the synopsis.

Types

- Functional and round-trip testing, including edge sizes and mixed-content files.
- Regression testing: the full automated suite runs on every push in Jenkins.
- Robustness testing: corrupted, truncated and malformed archives; fuzzing.
- Performance testing: thread scaling and comparison with gzip/DEFLATE and Zstandard.
- Usability testing: walkthrough of the web UI with a non-technical user, checking plain-language messages.
- Black-box testing (through CLI, API and UI) and white-box testing (unit and branch coverage).

Entry criteria: stable build from CI, test data corpus ready, test environment ready, module interfaces frozen for the phase being tested.

Exit criteria:

- 100% of planned test cases executed.
- No open critical or high defects.
- Whole corpus round-trips with identical SHA-256.
- Unit test coverage of at least 80% lines and 70% branches on core modules (target, to be confirmed with the guide).
- SonarQube quality gate passes; all acceptance criteria satisfied.

5.1 Security Validation

- Path traversal: archives with ../ or absolute paths must be rejected (TC-SEC-01)
- Denial of service: archives declaring huge sizes or counts are rejected before allocation (TC-SEC-02)
- API exposure: server reachable from localhost only, Origin and token checks (TC-SEC-03)
- Input validation of every API path and parameter (TC-SEC-04)
- Fuzzing of the archive reader with mutated archives (TC-NFR-03), run under ASan/UBSan
- Static analysis with SonarQube and dependency check in CI (TC-SEC-05)
- Penetration-style manual review of the HTTP API before the final demo (planned in Phase 5)
- No file contents or secrets in logs

6. Test Environment

Hardware: multi-core laptops/desktops used by the team (at least 4 cores for thread-scaling tests); a single-core configuration (taskset or a container limited to one CPU) for the single-core check.

Software: GCC/Clang with C++17, CMake, Docker (Ubuntu image used by Jenkins), Chrome and Firefox for UI tests, UATC core, CLI and server builds.

Tools: GoogleTest and CTest (unit/integration), gcov/lcov (code and branch coverage), Selenium (UI automation), curl/Postman (API), Jenkins (CI/CD), SonarQube (static analysis), ASan/UBSan/TSan

(sanitizers), libFuzzer or AFL (fuzzing), custom benchmark harness with gzip and zstd command-line tools, GitHub Issues/Jira (defect tracking).

Test data (corpus):

Category	Examples	Purpose
Text and source code	Plain-text books, log files, a source-code repository checkout	Text-like redundancy; Byte-shuffle/RLE choices
Numeric data	CSV time series, binary arrays of int32 / float64, sorted IDs	Delta and XOR transforms
Structured binary	Database dumps, BMP/WAV, executables	Repeated fields and layouts
Already compressed / encrypted	ZIP, JPEG, MP4, random bytes	Probe and store decision
Synthetic	All zeros, repeating pattern, ramp, mixed sections	Predictable expected behaviour
Size edge cases	0 bytes, 1 byte, block size -1 / exact / +1, 256 MiB+ file	Boundaries and scaling
Directory trees	Nested folders, thousands of tiny files, Unicode names, empty folders	Enumeration and structure
Damaged archives	Bit-flipped, truncated, wrong magic, future version, path traversal, oversized fields	Error handling and security

7. Test Schedule

Testing follows the six project phases in the SRS (Appendix B). Calendar dates are to be filled in from the course schedule.

Phase	Testing activity	Target date
Phase 1	Test strategy and this STP; repository, CI pipeline and test framework set up	TBD
Phase 2	Unit and integration tests for MVP: file layer, archive, baseline coder, checksums, extraction	TBD
Phase 3	Unit tests for probe, transforms, selection and store decision; regression suite grows	TBD
Phase 4	Thread pool and determinism tests; performance and benchmark runs; TSan	TBD
Phase 5	Corruption, edge-case, fuzz and security tests; API and UI tests; coverage gap closing	TBD
Phase 6	System validation on full corpus, comparative benchmarks, acceptance run, Test Summary Report	TBD

8. Test Deliverables

- Test Plan (this document)
- Test cases (manual and automated) and Requirements Traceability Matrix
- Test scripts and test data corpus
- Unit test report with code and branch coverage
- Test execution logs (CI)
- Defect reports
- Benchmark report

- Test Summary Report and maintenance plan

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Manasa K	Maintain this plan and the traceability matrix, coordinate integration, API and UI tests, produce the Test Summary Report
Test Engineer (core)	Sai Rishi Gangarapu	Tests for probe, transforms, selection, entropy coder, store decision (TC-CORE)
Test Engineer (archive)	S Chakshu	Tests for file layer, archive format, integrity and error cases (TC-FS, TC-ARC, TC-INT, TC-ERR, TC-SEC-01/02)
Test Engineer (performance)	Sampada K	Tests for thread pool, determinism, benchmarks (TC-PAR, TC-NFR-02)
Developer	Module owners	Fix defects in own modules; every member writes unit tests for their own functionality
Product Owner	Sai Rishi Gangarapu	Approve test results, sign off release readiness
Project Guide	Dr. Sapna V M	Review and approve the plan and results

10. Risks and Mitigation

Risk	Mitigation
Late or unstable builds	Frequent small merges; Jenkins runs the suite on each push; smoke tests first
Not enough time for hardening	Write tests alongside code each phase; fuzz and security work planned in Phase 5
Performance numbers vary between machines	Record machine details; repeat runs; report medians; compare relative not absolute
Flaky parallel tests	Determinism checks, TSan, fixed seeds, repeated runs
UI tests are fragile	Stable element IDs in the frontend; keep the UI thin and test logic at the API level
Test data corpus is not representative	Cover all categories in Section 6; add cases when defects are found

11. Assumptions & Dependencies

- Interfaces in SAD Section 4.3 are stable enough to write tests against from Phase 2
- Test corpus is available before execution starts
- Every team member has a working C++/CMake toolchain, Git and Docker
- Jenkins, SonarQube and GitHub are available to the team
- gzip and zstd command-line tools are available for benchmarking

12. Suspension & Resumption Criteria

Suspend testing if:

- The build is broken or so unstable that more than 30% of planned test cases are blocked

- A defect could destroy or corrupt test data or the shared environment
- The CI environment is unavailable for more than 4 hours

Resume testing when:

- Blocking defects are fixed and a new stable build passes smoke tests
- The environment is restored

13. Test Case Management & Traceability

Test cases are stored in the repository under tests/ and named with the IDs below. Each requirement maps to at least one test case; the full case list is in Appendix A. This table is the STP part of the Requirements Traceability Matrix (SRS Appendix C: Test Case ID and System Test Case ID columns).

Requirement	Description	Test case IDs
FR-01	File and directory input	TC-FS-01, TC-FS-02, TC-FS-03, TC-CLI-01
FR-02	Archive creation	TC-ARC-01
FR-03	Extraction, byte-for-byte	TC-ARC-02, TC-ARC-03, TC-ARC-04, TC-CLI-01, TC-SEC-01
FR-04	Per-block statistics	TC-CORE-01
FR-05	Compressibility probe	TC-CORE-02
FR-06	Adaptive transform selection	TC-CORE-03, TC-CORE-04, TC-CORE-08
FR-07	Entropy coding	TC-CORE-05
FR-08	Store-or-compress decision	TC-CORE-06, TC-CORE-07, TC-CORE-08
FR-09	Integrity verification	TC-INT-01, TC-INT-02, TC-VER-01
FR-10	Parallel execution, identical output	TC-PAR-01, TC-PAR-02, TC-PAR-03
FR-11	Archive inspection	TC-INFO-01, TC-VER-01, TC-CLI-01
FR-12	Error detection and reporting	TC-FS-03, TC-ERR-01, TC-ERR-02, TC-ERR-03, TC-ERR-04, TC-SEC-01, TC-SEC-02
FR-13	Web-based interface	TC-UI-01, TC-UI-02
FR-14	Progress and status display	TC-API-03, TC-UI-03, TC-UI-04
FR-15	Frontend-backend API	TC-API-01, TC-API-02, TC-API-03, TC-API-04, TC-SEC-03, TC-SEC-04
NFR-01	Correctness: byte-for-byte reconstruction	TC-ARC-02, TC-ARC-03, TC-ARC-04, TC-CORE-03, TC-CORE-05, TC-NFR-01
NFR-02	Performance: throughput scales with threads; honest comparison with gzip/Zstandard	TC-PAR-04, TC-PAR-05, TC-NFR-02
NFR-03	Reliability: corruption always detected	TC-INT-01, TC-INT-02, TC-PAR-02, TC-NFR-03, TC-SEC-02, TC-SEC-05
NFR-04	Usability: simple CLI and web UI	TC-UI-04
NFR-05	Maintainability: modular, add a transform without redesign	TC-NFR-05
NFR-06	Portability: standard C++17	TC-NFR-06
NFR-07	Observability: ratio, time, throughput, per-block transform	TC-NFR-07
NFR-08	Responsiveness: UI usable during long jobs	TC-UI-05

14. Test Metrics & Reporting

Metrics collected:

- Percentage of test cases executed, passed and failed
- Line and branch coverage per module (gcov/lcov)
- Requirement coverage (requirements with at least one passing test)
- Defect density and defect ageing
- Fuzzing: cases run and crashes found
- Benchmark results: ratio, speed and resource use per corpus category

Reports: CI test and coverage report per build, weekly status to the guide, benchmark report, and the final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
QA Lead	Manasa K	
Dev Lead / Product Owner	Sai Rishi Gangarapu	
Team Member	S Chakshu	
Team Member	Sampada K	
Project Guide	Dr. Sapna V M	

Appendix A: Test Case Catalogue

Levels: Unit, Integration, System, Performance, Security. Expected results refer to SRS/SAD behaviour; thresholds in performance cases are recorded, not pass/failed, unless the guide sets a target.

ID	Req.	Test description	Expected result	Level
TC-FS-01	FR-01	Compress a single file path	Exactly one file entry enumerated; size and name recorded	Unit
TC-FS-02	FR-01	Compress a nested directory tree with empty folders and Unicode file names	All files and folders enumerated; structure preserved	Unit
TC-FS-03	FR-01, FR-12	Input path does not exist or is unreadable	E_NOT_FOUND / E_PERMISSION returned; no crash; no partial archive left	Unit
TC-ARC-01	FR-02	Create an archive and inspect its structure	Magic, version, global metadata, file table, block index and payloads all present and consistent	Integration
TC-ARC-02	FR-03, NFR-01	Round trip of a single file (compress, extract, compare SHA-256)	Extracted file is byte-identical	Integration
TC-ARC-03	FR-03, NFR-01	Round trip of a directory tree	Every file identical; directory structure identical	System
TC-ARC-04	FR-03, NFR-01	Edge sizes: empty file, 1 byte, exactly one block, block size +/- 1, many tiny files	All round-trip correctly	Unit
TC-CORE-01	FR-04	Compute statistics on known blocks (all zeros, uniform random, English text)	Statistics match hand-computed values	Unit
TC-CORE-02	FR-05	Probe on text, zeros, random and already-compressed data	Random / compressed flagged incompressible; text and zeros flagged compressible	Unit
TC-CORE-03	FR-06, NFR-01	Round trip for every transform (None, Delta, XOR, Byte-shuffle, RLE) on parameterised inputs	decode(encode(x)) == x for all inputs, including empty	Unit
TC-CORE-04	FR-06	Selection on crafted blocks (numeric ramp, long runs, repeated records, mixed)	Expected transform chosen; ties resolve to the lowest transform ID	Unit
TC-CORE-05	FR-07, NFR-01	Entropy coder round trip on random, skewed and single-symbol data	Decoded data identical to input	Unit
TC-CORE-06	FR-08	Block of random bytes	Block stored unchanged; stored length equals original length	Unit
TC-CORE-07	FR-08	Block whose coded size plus metadata is not smaller than original	Store decision taken; archive not larger than necessary	Unit
TC-CORE-08	FR-06, FR-08	Mixed file (text + zeros + random sections)	Different blocks of one file get different transforms / store; round trip correct	Integration
TC-INT-01	FR-09, NFR-03	Flip one byte inside a block payload, then verify and extract	E_BLOCK_CHECKSUM with block index; no silently wrong output	Integration
TC-INT-02	FR-09, NFR-03	Corrupt archive-level checksum / footer	E_ARCHIVE_CHECKSUM reported	Integration
TC-ERR-01	FR-12	Truncate archive at several offsets	E_MALFORMED_ARCHIVE or E_MISSING_BLOCK as appropriate; no crash or hang	Integration
TC-ERR-02	FR-12	Set format version to a future major version	E_UNSUPPORTED_VERSION	Unit
TC-ERR-03	FR-12	Wrong magic bytes / non-archive file	E_MALFORMED_ARCHIVE	Unit
TC-ERR-04	FR-12	Trigger each error case above and compare	Each case gives a distinct code and message	System
TC-PAR-01	FR-10	Compress the same input with workers = 1 and workers = 8	Archives are byte-identical	System

ID	Req.	Test description	Expected result	Level
TC-PAR-02	FR-10, NFR-03	Repeat the 8-worker run 20 times; run the suite under ThreadSanitizer	Identical archive every time; no data races reported	System
TC-PAR-03	FR-10	Thread pool unit tests: all tasks run exactly once, clean shutdown, exception in a task is reported	Behaviour as described; no deadlock or leak	Unit
TC-PAR-04	NFR-02	Large input (>= 256 MiB) with 1, 2, 4, 8 workers	Throughput grows with workers up to I/O limit; numbers recorded in report	Performance
TC-PAR-05	NFR-02	Very small input	Serial path used; no slowdown from thread start-up	Performance
TC-INFO-01	FR-11	Run info on a known archive	Size, file count, block count, transform usage and ratio match expected values; no files extracted	Integration
TC-VER-01	FR-09, FR-11	Run verify on an intact archive	Success message, exit code 0, nothing written to disk	Integration
TC-CLI-01	FR-01, FR-03, FR-11	Run compress, extract, info, verify, benchmark and --help	All sub-commands exist and behave as in SRS Section 4; exit codes as in SAD 4.3.4	System
TC-API-01	FR-15	POST /api/jobs/compress then poll GET /api/jobs/{id}	202 with jobId; state moves queued, running, completed; result has statistics	Integration
TC-API-02	FR-15	POST with missing or invalid fields	400 with error object (code, message)	Integration
TC-API-03	FR-15, FR-14	Start a job whose destination is not writable	Job state failed with E_PERMISSION or E_IO in the error object	Integration
TC-API-04	FR-15	GET /api/fs/list with a path outside the allowed root	403 E_PERMISSION; no listing returned	Integration
TC-UI-01	FR-13	In a browser select a folder and press Compress	Job starts; archive is created at chosen destination	System
TC-UI-02	FR-13	In a browser select an archive and extract it	Files restored and identical to original	System
TC-UI-03	FR-14	Watch a long job in the UI	Progress bar updates; results screen shows size, ratio, time and transform breakdown	System
TC-UI-04	FR-14, NFR-04	Force failures (bad archive, unwritable folder)	Plain-language message shown; no stack trace or raw error code	System
TC-UI-05	NFR-08	Interact with the page while a large job runs	Page stays responsive; polling continues	System
TC-NFR-01	NFR-01	Full corpus round trip (all data categories in Section 6)	All files SHA-256 identical to originals	System
TC-NFR-02	NFR-02	Run the benchmark against gzip/DEFLATE and Zstandard on the sample set, twice	Repeatable report with ratio, speed and resource use; runs agree within tolerance	Performance
TC-NFR-03	NFR-03	Fuzz the archive reader (random mutations, 10,000+ cases)	No crash, hang or silent wrong output; every bad input gives an error	Security
TC-NFR-05	NFR-05	Add a dummy test-only transform with a new ID	It is selected and round-trips without editing archive, API, frontend or CLI code	Integration
TC-NFR-06	NFR-06	Build and run the unit tests in the Docker (Linux) image and on a second OS used by the team	Both builds succeed; same test results	System
TC-NFR-07	NFR-07	Compress a known input and read the statistics	Ratio, elapsed time, throughput and per-block transform are reported	Integration
TC-SEC-01	FR-03, FR-12	Archive containing entry path ../../evil.txt or an absolute path	E_PATH_TRAVERSAL; nothing written outside destination	Security
TC-SEC-02	FR-12, NFR-03	Index declares an absurd block length / block count	Rejected before allocation; bounded memory	Security
TC-SEC-03	FR-15	Connect to the API from another host / wrong Origin / missing token	Connection refused or 403	Security

ID	Req.	Test description	Expected result	Level
TC-SEC-04	FR-15	Send path values with .. segments or odd encodings to the API	Normalised and checked; access outside allowed root refused	Security
TC-SEC-05	NFR-03	SonarQube scan and dependency check in CI	No open blocker or critical vulnerabilities; quality gate passes	Security